LAB 6 – ETL/ELT – DLT & DBT

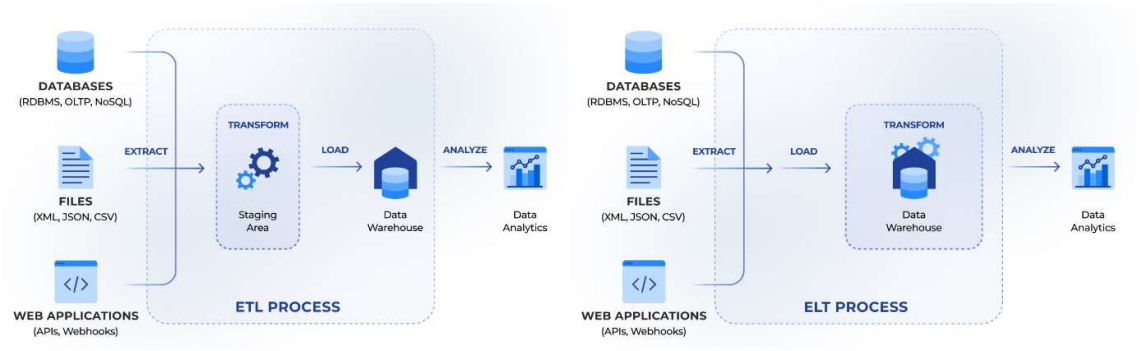
CONTENT

- SSIS Introduction:
  - Basic Transformations
    - https://docs.microsoft.com/en-us/sql/integration-services/data-flow/transformations/integration-services-transformations?view=sql-server-2017
      - Focus on Aggregate, Conditional Split, Derived Column, Lookup, Merge, Merge Join, Multicast, Pivot, Row Count, Sort, Union all

ETL I ELT

*The fundamental difference between **ETL and ELT** data processing patterns lies in where the data transformation occurs.*

- ***ETL (Extract-Transform-Load):** Transformations happen in a dedicated **staging area** before the data ever reaches the warehouse. Data is cleaned and reconciled in this intermediate environment, then loaded into the target storage for analysis.*
- ***ELT (Extract-Load-Transform):** Raw data is loaded directly into the target warehouse first. Transformations are then performed within the warehouse itself using SQL or other processing engines.*



(https://www.altamira.ai/blog/etl-vs-elt/)

In traditional ETL (Extract-Transform-Load), transformations take place before the data is loaded into the data warehouse. In this approach, data is extracted from external sources and loaded into a dedicated intermediate environment – known as a staging environment. In this environment, data transformation processes are carried out, including data cleansing and reconciliation. The appropriately prepared data is then loaded into the target data storage structures, usually a data warehouse, for the purposes of analysis, including OLAP. In the alternative ELT (Extract-Load-Transform), raw data is loaded directly into the data warehouse (e.g. DuckDB, Snowflake), and only there is it transformed using SQL or other data processing solutions.

*When building modern data pipelines, **dlt** and **dbt** often work together to handle different stages of the process.*

- ***dlt (data load tool)***

  *This tool focuses on the **Extract** and **Load** portions of the pipeline. It is a Python library designed to move data from various sources (like SQL Server) to a destination. It automatically manages complex tasks like **schema evolution** and **data typing**.*

- ***dbt Core (data build tool)***

  *This tool is the industry standard for the **Transform** stage. It allows developers to write transformations as **SQL SELECT statements** called models. It manages the dependencies between these models and generates documentation automatically.*

- ***Target Environment: DuckDB***

*In many modern setups, **DuckDB** serves as the destination warehouse. It is an in-process analytical database that runs locally as a file, making it highly efficient for BI analysis and development.*

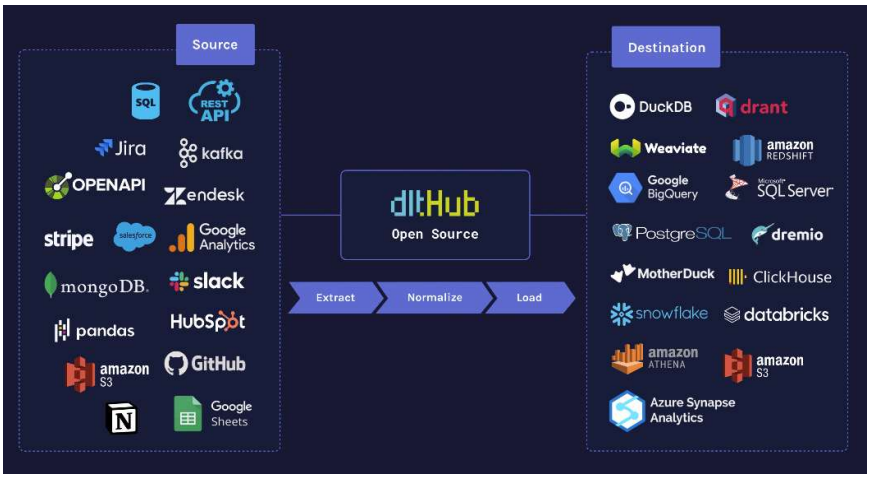
***Technical Note:** Due to package compatibility issues (specifically pyodbc for SQL Server), it is currently recommended to use **Python 3.12** in a virtual environment rather than Python 3.13 when working with dbt-core.*

In the following section, we will focus on ETL solutions, for which we will use a range of additional tools:

- *dlt (data load tool): A Python library used to transfer data from various sources (e.g. SQL Server) to destination systems. It automatically handles data schemas and data typing.*
- *dbt Core (data build tool): An industry standard for data transformation within a data warehouse. It allows you to write SQL code in the form of models (SELECT), and manages dependencies and documentation.*
- *DuckDB: A fast, in-process analytical database (runs locally as a file), ideal for learning and quick BI analyses.*

DLT – EXTRACTION

DLT is an open-source Python library that automates the most tedious parts of the ELT process. It allows you to automatically transform data from a variety of available sources (e.g. APIs, PostgreSQL databases) into a real-time updated dataset stored in a user-selected destination (e.g. BigQuery, Deltalake, DuckDB). You can easily integrate your own sources, provided they are compatible with dlt (e.g. JSON, Python lists and dictionaries, pandas Dataframes), and dlt will be able to automatically identify the schema and transfer the data to the destination.



(https://dlthub.com/docs/reference/explainers/how-dlt-works#the-three-phases)

This solution offers a few key features:

- *Schema Evolution: If a new column appears in the source SQL Server, dlt detects this and automatically adds it to the target database (Staging or DuckDB).*
- *State Management: dlt remembers which data has already been loaded (so-called incremental loading), allowing only new records to be retrieved efficiently.*
- *Multiple Destinations: The same source code can send data to SQL Server, DuckDB, Snowflake or BigQuery simply by changing a parameter.*

THE THREE PHASES OF THE PROCESS IN DLT:

During the extraction phase, the dlt tool fully retrieves data from the sources to the hard drive into a new load package, which will be assigned a unique identifier and will contain the raw data received from the sources. Additionally, schema hints can be provided to define the data types of certain columns or to add a primary key and unique indexes. This phase can also be controlled by limiting the number of items extracted in a single pass,

using incremental cursor fields, and optimising performance through parallelism. You can also apply filters and maps to encrypt or remove personal data, as well as use transformers to create derived data.

Normalisation requires the extraction phase to be complete and will not perform any actions if no data has been extracted. During the normalisation phase, dlt checks and normalises the data and calculates a schema corresponding to the input data. The schema will automatically evolve to adapt to any future changes in the source data, such as new columns or tables. dlt will also unpack nested data structures into child tables and create variant columns if the detected values do not match the schema calculated during the previous run. The result of the normalisation phase is an updated load package containing normalised data in a format understandable to the destination, along with a complete schema that can be used to migrate data to the destination. The normalisation phase can be controlled, for example by defining the permitted level of input data nesting, applying schema conventions governing schema evolution, and specifying how to handle rows that do not match the schema. Performance settings are also available.

The Load operation requires the normalisation phase to have been completed and will not perform any actions if there is no normalised data. During the load phase, dlt first runs schema migrations at the target, if necessary, and then loads data into that location. dlt loads data in smaller chunks, known as load tasks, to enable parallel processing of large data sets. If the connection to the target fails, the pipeline can be safely restarted, and dlt will continue loading all load tasks from the current load batch. dlt will also create special tables in which the internal dlt schema, information about all load batches, and certain state information are stored; this information is used, amongst other things, by incremental operations to restore the incremental state from a previous run on a different machine. Some methods of controlling the loading phase involve using different write_dispositions to overwrite data at the destination, simply append to it, or merge based on specific merge keys, which can be configured for each table. For some destinations, a remote staging dataset at the bucket provider can be used, and dlt even supports modern open table formats such as deltables and iceberg, and reverse ETL is also possible.

For our purposes, we will use dlt as the data extraction mechanism. In dlt, the extraction process from SQL databases is based on the verified sql_database source. This utilises the SQLAlchemy engine underneath, which ensures compatibility with most SQL dialects. The extraction process itself is carried out in the following steps:

1. *Initialisation: dlt connects to the source (using a connection string).*
2. *Inspection: The tool reads the source's metadata (table names, column types, primary keys) – in dlt terminology, this is called Normalise.*
3. *Iteration: Data is retrieved in chunks, which prevents RAM overflow when dealing with large tables.*
4. *Incremental Declaration: We can define a 'cursor' column (e.g. ModifiedDate), which allows dlt to retrieve only data newer than that from the last successful run.*

Unlike heavy-duty ETL tools with a graphical interface, dlt is a Python library that can be used directly within your code. The process in dlt is divided into three main components:

1. *Source: A logical group of resources (e.g. "AdventureWorks database" or "Facebook API").*
2. *Resource: A specific data stream (e.g. the Sales table or the result of an API query). More specifically, a resource is a function (the @dlt.resource decorator) that returns data sequentially.*
3. *Pipeline: An engine that connects the source to the destination, manages the schema and state. It transfers data to the destination, handles dlt sources or resources, as well as any iterable objects. Once started, all resources are processed and the data is loaded into the destination.*

QUICK START – DLT

Below is a link to a brief introduction to what DLT is, how to run a simple data processing pipeline using sample data, and how to view the loaded data:

> *https://colab.research.google.com/github/dlt-hub/dlt/blob/master/docs/education/dlt-fundamentals-course/lesson_1_quick_start.ipynb*

CONFIGURATION I: SQL SERVER (SOURCE) -> SQL SERVER (STAGING)

This scenario is typical for enterprises where raw data is copied from production systems (ERP/CRM) to a dedicated staging database on the same or a different MSSQL server prior to further processing.

Why this two-stage process?

1. SQL Server -> SQL Server: Takes the load off the production system. Extraction is fast and secure.

2. SQL Server -> DuckDB: DuckDB offers columnar performance, which for analytical queries (JOINs in a star schema) is orders of magnitude higher than in traditional SQL Server, drastically speeding up the performance of BI dashboards.

First, we need to install DLT and the necessary libraries for connecting to the SQL Server database:

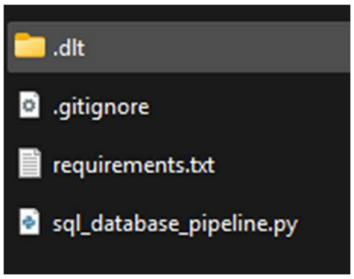
```
pip install dlt[mssql], pymssql
```

An ODBC driver is required (e.g. ODBC Driver 17/18 for SQL Server); the driver is available on the Microsoft website – https://learn.microsoft.com/en-us/sql/connect/odbc/download-odbc-driver-for-sql-server?view=sql-server-ver17

We create the basic project structure (in the current directory), where the source is the SQL database (sql_database) and the destination is SQL Server (mssql). To do this, we use the ready-made dlt scripts:

```
dlt init sql_database mssql
```

You can view the list of available sources using the command `dlt init -l`, and the list of available destinations using the command `dlt init --list-destinations`. A simple project structure will be created:




```
.
├── requirements.txt
├── rest_api_test_pipeline.py
├── .dlt
│   ├── config.toml
│   ├── secrets.toml
```

Next, we install the required dependencies based on the requirements file we created:

```
pip install -r requirements.txt
```

Configuring Credentials (.dlt/secrets.toml) – For security reasons, we store the login details in the secrets.toml file:

```
# Source (Production)
[sources.sql_database]
credentials="mssql+pyodbc://user:password@host/AdventureWorks?driver=
ODBC+Driver+18+for+SQL+Server&TrustServerCertificate=yes"
schema="Sales"
# Destination (Staging)
[destination.mssql.credentials]
database = "StagingDB"
password = "password"
username = "user"
host = "host"
port = 1433
driver = "ODBC Driver 18 for SQL Server"
[destination.mssql.credentials.query]
# trust self-signed SSL certificates
TrustServerCertificate="yes"
# require SSL connection
Encrypt="no"
# send large string as VARCHAR, not legacy TEXT
LongAsMax="yes"
```

The second configuration file, config.toml, contains the basic configuration, which does not include any sensitive data. Here, we define additional runtime parameters, such as logging levels, or parameters specific to a particular connection type, e.g. database names.

We can now move on to implementing the pipeline in DLT. As mentioned earlier, a pipeline is simply a Python object that we create using the relevant extensions of the dlt library. In the example below, we transfer two tables, SalesOrderHeader and SalesOrderDetail, from the source server to the target server. We achieve this by creating an object representing the source (limited to specific resources – tables) and creating a pipeline (ultimately to mssql). Finally, we can connect the source to the pipeline and run the extraction and data loading (in the appropriate loading mode):

```
import dlt
from dlt.sources.sql_database import sql_database

def run_staging_load():
    # 1. We define the source – we select a specific scheme 'Sales'
                                                source                    =
sql_database(schema="Sales").with_resources("SalesOrderHeader")
                                               source2                    =
sql_database(schema="Production").with_resources("Product")

    # 2. We are creating a pipeline to the target SQL Server    pipeline
= dlt.pipeline(
        pipeline_name="sql_to_staging",
        destination="mssql",
        dataset_name="raw" # Schemat w bazie stagingowej
    )

    # 3. The 'append' mode allows data to be appended incrementally
    info = pipeline.run(source, write_disposition="append")
    print(info)
    info2 = pipeline.run(source2, write_disposition="append")
    print(info2)

if __name__ == "__main__":
    run_staging_load()
```

In short, simply call the `sql_database()` function to connect to the database and retrieve the source data. All configuration details are specified in TOML files, so this function can be called without providing any parameters. In the example above, we provide an additional parameter that specifies the schema of the resource being retrieved – in this case, a table.

Next, we can define a simple dlt.pipeline() pipeline, which is connected to a specific destination. In the example above, this is an SQL Server.

Finally, we can run individual pipelines using pipeline.run(), where we specify the source and the data loading method. Loading modes are defined within the `write_disposition` parameter. This determines how data is written to the table. The default value is "append".

- *The "append" option always appends new data to the end of the table.*
- *The "replace" option replaces existing data with new data.*
- *The "merge" option removes duplicates and merges data based on the `primary_key` and `merge_key` specifications.*

When performing incremental loading of data, to avoid transferring the entire AdventureWorks dataset on every run, DLT allows you to define a change tracking field:

```
# Przykład dla pojedynczej tabeli
source = sql_database()
source.SalesOrderHeader.apply_hints(
    incremental=dlt.sources.incremental("ModifiedDate",
    initial_value="2020-01-01T00:00:00Z")
)
```

DLT also allows for basic monitoring and logging of each run. In short, `pipeline.run()` returns a `LoadInfo` object, which contains:

- *Statistics on the number of rows loaded.*
- *Information about schema changes (e.g. added columns).*
- *Details regarding any errors in specific data batches.*

Additionally, we can view the history of calls and their status via the web interface (uses Streamlit):

```
dlt pipeline sql_to_staging show
#sql_to_staging is the name of the pipeline you want to investigate
```

However, this requires the installation of additional packages within dlt[workspace].

We can then run the pipeline implemented in Python code, and the entire defined task should be executed, with the relevant tables and data created in the StagingDB database.

CONFIGURATION II: SQL SERVER (STAGING) -> DUCKDB (FINAL)

As an alternative, or following the cleaning and transformation process (often carried out by dbt within SQL Server), the data can be sent to DuckDB – a local, ultra-fast analytical database that serves as a source for BI reports or tools such as Streamlit.

In the alternative solution, we implement an ELT approach, where data goes directly to DuckDB and is prepared for analysis there.

Integrating DuckDB into the entire process requires installing the relevant libraries and DuckDB itself:

```
pip install dlt[duckdb] duckdb
```

With DuckDB, configuring credentials is simpler, as the database is a file.

```
# Destination (Local DuckDB)
[destination.duckdb.credentials]
database = "bi_analytics.duckdb" # Path to the database file
```

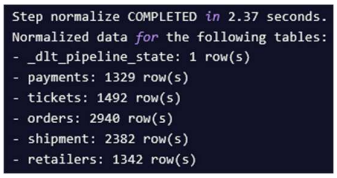
Stream implementation:

```
import dlt
from dlt.sources.sql_database import sql_database

def run_final_load():
    # We retrieve the already processed tables from the staging database
    source = sql_database().with_resources("dim_product",
"dim_customer", "fct_sales")

    pipeline = dlt.pipeline(
        pipeline_name="staging_to_duckdb",
        destination="duckdb",
        dataset_name="gold_zone"
    )

    # We use 'replace' to ensure we always have the latest version of
the star schema in DuckDB
    info = pipeline.run(source, write_disposition="replace")
    print(info)
```

To check the number of rows loaded for each table, you can use the following command:

```
dlt pipeline <nazwa_pipeline> trace
```

This command will display the names of the loaded tables and the number of rows in each. For example:



DBT – TRANSFORMATIONS

In the traditional ETL (Extract, Transform, Load) process, transformations often took place before the data was loaded into the data warehouse. dbt changes this approach by adopting ELT. It assumes that the data is already in the data warehouse, and dbt is solely responsible for the T (Transform) layer. Although in our example we will use dbt in the traditional ETL sense, where dbt will transfer data twice between the source and the staging layer, and between the staging layer and the target.

We need to clarify an important terminological point: dbt Core is not used to transfer data from external sources (e.g. APIs or Excel files) into the database. There are other tools for this, such as dlt, Airbyte, Fivetran or simple Python scripts. In dbt, 'extraction' (or rather, its definition) involves mapping raw data that is already in your SQL Server so that dbt can manage it.

Key concepts of dbt:

- *Models: These are simply .sql (SELECT) files that dbt converts into tables or views.*
- *Modularity: SQL code can be broken down into smaller chunks and reused multiple times thanks to the Jinja engine.*
- *Version control: dbt treats SQL code like software code (integration with Git).*
- *Testing: Automatic data quality verification (e.g. whether primary keys are unique).*

DBT is responsible for the T (Transform). The most important thing to realize is that dbt does not process your data. It is not a calculation engine; it is a code generator and orchestrator. When you write a dbt model, you only write a standard SELECT statement. You don't write CREATE TABLE or INSERT INTO. dbt's role is to wrap your SELECT statement in the necessary Data Definition Language (DDL) or Data Manipulation Language (DML) for your specific database (MSSQL, Snowflake, etc.). When you execute the dbt run command couple of things happen. At first, it is the ***compilation –*** *dbt reads your .sql files and replaces the Jinja templates (like {{ ref('my_table') }}) with actual table names from your database. Then it is the* ***generation –*** *dbt adds the heavy lifting code. If your model is configured as a "table," dbt generates a SQL statement like CREATE OR REPLACE TABLE destination_schema.my_model AS (..). Next, dbt sends this massive block of SQL to your SQL Server – as a massive execution step. Finally, your database server does the actual work of moving bytes and calculating numbers. dbt simply waits for a "Success" or "Error" message from the database.*

Because dbt only uses SELECT statements, it offers several major advantages. First, it is ***idempotency****, as you can run dbt repeatedly. Since it manages the DROP/CREATE or TRUNCATE/INSERT logic, it always results in the same clean state. Just be aware that your naming must be on point (look out for small/capital letters discrepancies). Next, it is the* ***dependency management****, as dbt looks at your ref() tags to build a Directed Acyclic Graph (DAG). It knows it must finish building stg_sales before it can start building fct_orders. Finnally, it is the easy* ***environment switching.*** *The same SELECT statement can be run against a dev schema or a prod schema just by changing your profiles.yml or by adding a --target prefix to run command.*

In short, you can think of dbt as a compiler for SQL. Just as a programmer writes high-level code and the compiler turns it into machine instructions, a data engineer writes clean SELECT statements and dbt turns them into complex database administrative commands.

DBT - INSTALLATION

Installation: dbt requires a dedicated driver to 'communicate' with your AdventureWorks database.

```
pip install dbt-core dbt-sqlserver dbt-duckdb
```

Next, we can create a new project:

```
dbt init project_name
```

During setup, you will be asked for your SQL server details (Host, Port, User, Password, Database). The basic project folder structure and core configuration files will be created.

In dbt Core, YAML files (.yml) are used for everything other than data transformation: from project configuration and defining data sources to tests and documentation.

Main file: dbt_project.yml

This is where you specify global parameters. Here, you can define that all models in the marts/ folder should be tables by default, and those in staging/ should be views. For the purposes of our tasks, we will use the default folder and file structure.

The profiles.yml file

This stores database login details and dbt project configurations, including:

Connection details — account IDs, hosts, ports and authentication credentials.

Target environment definitions — defining different environments (dev, staging, prod) within a single profile.

Default environment — setting the environment to be used by default.

Execution parameters — number of threads, timeouts and retry settings.

Authentication data separation — protects sensitive information from version control.

Example:

```
testsbi:
  target: dev
  outputs:
    dev:
      type: sqlserver
      driver: 'ODBC Driver 18 for SQL Server'
      server: host
      port: 1433
      database: dbname
      schema: schemaname
      user: username
      password: password
      encrypt: false
```

For more details look at:

- https://docs.getdbt.com/docs/local/connection-profiles
- https://docs.getdbt.com/docs/local/connect-data-platform/mssql-setup?version=1.11#standard-sql-server-authentication

Schema files – schema.yml

This allows you to include additional database documentation in a file (e.g. models/staging/schema.yml), which enables you to describe models in detail and share this information with the team. It is important that the reference name matches the name of the model file.

```
version: 2
models:
  - name: customers
    description: One record per customer
    columns:
      - name: customer_id
        description: Primary key
        data_tests:
          - unique
```

```
        - not_null
    - name: first_order_date
      description: NULL when a customer has not yet placed an order.
- name: stg_customers
  …
```

Source files – X.yml

These allow you to define sources, which involves mapping raw data that is already stored in your SQL Server so that dbt can manage it. To do this, we create a file (e.g. models/staging/src_adventureworks.yml) that tells dbt: "In this database, in this schema, there are tables that I will be using."

```
version: 2
sources:
  - name: raw_aw # Nazwa referencyjna w dbt
    database: AdventureWorks2022 # Nazwa bazy w SQL Server
    schema: Sales # Schemat w bazie
    tables:
      - name: SalesOrderHeader
      - name: SalesOrderDetail
      - name: Customer
```

At the heart of dbt lies the way we 'describe' transformations using pure SQL code enhanced by the Jinja engine. So let's move on to the technique for defining transformations that will take us from raw tables to a finished star schema. However, remember that a dbt model is a SELECT SQL statement stored in the models/ directory. dbt converts these statements into a table or view, depending on the settings.

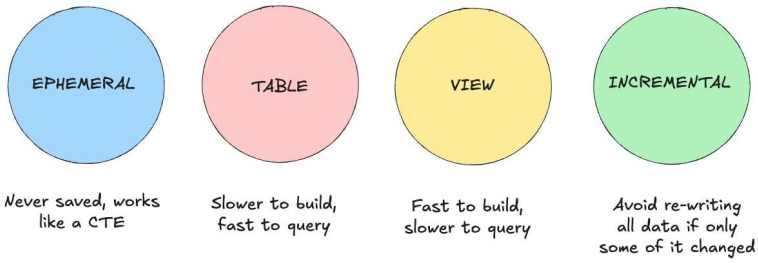
Once again, in dbt, a transformation is a model. A model is simply a .sql file located in the models/ folder. Each such file corresponds to a single table or a single view in the database. This means that when using dbt, you only need to prepare the SELECT statements, whilst dbt handles the entire DDL (Data Definition Language) wrapper. In short, dbt automatically executes the CREATE TABLE AS... or CREATE VIEW AS... commands.

A standard dbt model consists of two parts: a configuration block (optional) and an SQL query. Example: models/staging/stg_products.sql

```
{{ config(materialized='view') }} - Configuration details for storage
SELECT
    ProductID AS product_id,
    Name AS product_name,
    ProductNumber AS product_code,
    StandardCost AS unit_cost
FROM {{ source('adventureworks', 'Product') }}
-- Dynamic reference to the source
```

In dbt, you decide how the data should be physically stored in SQL Server using the `materialized` parameter:

- *view (default): The transformation is a view. Quick to build, always up to date, but may be slower with large joins.*
- *table: The transformation creates a physical table. Speeds up reads in BI, but requires a reload (dbt run).*
- *incremental: dbt only adds new records (advanced).*
- *ephemeral: The model does not appear in the database; it exists only as a subquery (CTE) within other models.*



(https://thepipeandtheline.substack.com/p/sql-to-dbt-guide-your-dbt-starter)

To ensure that transformations are modular and easy to maintain, dbt introduces two key features:

The ref() function – Creating lineage

This is the most important function in dbt. Instead of writing `FROM dbo.stg_products`, you write `FROM {{ ref('stg_products') }}`.

- *Why? dbt automatically builds a directed acyclic graph (DAG). It knows that it must first build table A in order to build table B.*
- *Advantage: You can change database schemas, and dbt will automatically ensure the paths are correct.*

source() function – Safe sources

Instead of manually entering source table names, you can define them in a single .yml file and refer to them later. This means that if the name of a source database changes, the update is relatively straightforward – the whole process is carried out in just one place in the configuration, rather than in every SQL file.

It is worth noting that ref() does three magical things: it detects dependencies and build order, points to the correct schema for each environment, and allows for the visualisation of neat dependency graphs.

In addition, by running:

```
dbt run --select orders
```

It is only possible to build a given model (orders) and its parent models.

DBT - TESTING

Without the testing phase, dbt would be nothing more than 'SQL on steroids'. It is automated testing that transforms ordinary scripts into a professional data engineering process, giving us the assurance that BI reports in Power BI or Tableau do not display incorrect figures. The Quality Assurance (QA) module in dbt Core, which rounds off the list of the tool's core capabilities.

In dbt, tests are nothing more than SQL queries that 'look for problems'. If a test query returns even a single record, the test fails.

We distinguish between two main types of tests:

Generic Tests (in .yml files)

These are ready-made test templates that we add to columns in configuration files. dbt Core offers four standard tests:

- *unique: checks that values in a column do not duplicate (e.g. Primary Key).*
- *not_null: checks that there are no empty values.*
- *accepted_values: checks that the data falls within a specified list (e.g. Status: 'Sended', 'Cancelled').*
- *relationships: checks referential integrity (whether the product_id in the facts exists in the dimension table).*

Example (models/marts/dim_product.yml):

```
version: 2
models:
  - name: dim_product
    columns:
      - name: product_id
        tests:
          - unique
          - not_null
      - name: category_name
        tests:
          - accepted_values:
              values:   ['Bikes',   'Components',   'Clothing',
'Accessories']
```

Singular Tests

If you have a specific business rule (e.g. "Total sales cannot be negative"), create an .sql file in the tests/ folder.

Example (tests/assert_sales_value_is_positive.sql):

```
SELECT
    order_id,
    total_revenue
FROM {{ ref('fact_sales') }}
WHERE total_revenue < 0 -- The test looks for errors, so SELECT
must return invalid records
```

We run the tests using the command:

- *dbt test – runs all defined tests.*
- *dbt test --select dim_product – tests only the selected model.*

EXAMPLE: TRANSFORMATION IN PRACTICE (THE STAR MODEL)

*Let's assume we are creating a Customers dimension table (**dim_customer**). We need to combine customer data with their personal information.*

Step 0: Configuration

*We must create all the basic configuration files, including checking **dbt_project.yml**, adding connection data within **profiles.yml**, and creating schema and source data files as needed. For the purposes of this tutorial, we will add the source definition file in Step 1.*

Step 1: Source Definition (models/staging/src_aw.yml)

```
version: 2
sources:
  - name: adventureworks
    tables:
      - name: Product
```

Note: The name within the sources block corresponds to the schema name in the database.

Step 2: Transformation (models/staging/stg_product.sql)

Next, we can define simple data processing based on the defined source data:

```
{{ config(materialized='view') }}
WITH raw_prod AS (
    SELECT * FROM {{ source('adventureworks', 'Product') }}
```

```
)
SELECT product_id, color
FROM raw_prod
```

Step 3: Transformation (models/marts/dim_product.sql)

Next, we can define the subsequent data transformation step based on the previously defined steps:

```
{{ config(materialized='table') }}
WITH stg_prod AS (
    SELECT * FROM {{ ref('stg_product') }}
)
SELECT product_id, color, 1 as new_col
FROM stg_prod
```

Note: The name used within ref() must match the filename of the previous data transformation step.

Step 4: Running

*Running **dbt (data build tool)** is a process that turns raw SQL code into a professionally managed data pipeline, allowing analysts to work much like software engineers. Remember that dbt focuses on the **transformation layer** within the ETL/ELT process. Run the script by calling:*

```
dbt run
```

*At this point, dbt compiles the code and executes it in the database. If you use **Jinja macros** (e.g., {{ ref('other_model') }}), dbt will automatically detect the execution order, creating a **DAG (Directed Acyclic Graph)**.*

Step 5: Generating Documentation

dbt automatically collects metadata about your tables, columns, and relationships. dbt docs generate

dbt will search your project and database schema, then generate JSON files (including manifest.json and catalog.json) in the target/ folder. These contain complete knowledge about your data structure.

Visualization (Local Server): To view the documentation as a website, use:

```
dbt docs serve
```

By default, a web portal will be available at http://localhost:8080 with basic information:

- **Lineage Graph (Data Origin Chart):** Shows a graphical path of the data—from source tables, through intermediate models, to final reports. This allows you to immediately see the impact of changing a single column at the beginning of the process.
- **Descriptions:** If you added descriptions in the .yml files, they will appear next to the tables and columns.
- **Tests:** Information regarding which quality tests (e.g., unique, not_null) are applied to the data.
- **SQL Code:** You can preview both the source code and the compiled code (the final SQL sent to the database).

DBT MODEL STRUCTURE

In the dbt transformation process, we typically use a layered approach:

1. *Staging (stg): Direct reference to raw_data, column name cleaning, simple type casting.*
2. *Intermediate (int): Joining tables (e.g. Header with Detail), reconciling business logic.*
3. *Mart (fct/dim): Final star schema tables.*

DBT AND DLT

For those interested, it is possible to run dbt pipelines within dlt:

https://dlthub.com/docs/dlt-ecosystem/transformations/dbt

ADDITIONAL ONLINE RESOURCES – DLT AND DBT

dlt:

- https://dlthub.com/docs/pipelines/x/load-data-with-python-from-x-to-mssql#2-configuring-your-source-and-destination-credentials
- https://dlthub.com/docs/pipelines/sql_database_mssql/load-data-with-python-from-sql_database_mssql-to-duckdb
- https://www.datacamp.com/tutorial/python-dlt

dbt:

- https://learn.getdbt.com/courses/dbt-fundamentals
- https://docs.getdbt.com/guides/duckdb
- https://docs.getdbt.com/guides/manual-install

DOCKER ENVIRONMENT FOR TESTING PURPOSES AND ASSIGNMENTS

On EPortal there is a prepared docker environment you can use to write your dlt and dbt scripts, or test some exemplar ones. This Docker environment is designed as a self-contained data platform. It separates the **storage** (SQL Server) from the **logic** (Data Tools), allowing you to simulate a professional modern data stack on your local machine.

The setup consists of two main services defined in your docker-compose.yaml:

- **sqlserver**: The **Target Warehouse**. This is where your data eventually lives. It persists data in the ./mssql_data folder, so your work isn't lost when you stop the containers.
- **data_tools**: The **Execution Engine**. This container is where you run your Python scripts (dlt) and your transformation models (dbt). It has the Microsoft ODBC Driver installed to communicate securely with the SQL Server.

To get your environment up and running, you will use **Docker Compose**. This tool reads your configuration file and starts both the SQL Server and the Data Tools container in the correct order, setting up the virtual network between them. Open your terminal in the directory where your docker-compose.yaml is located and run:

```
docker-compose up -d
```

- **up**: Tells Docker to build (if necessary) and start the containers.
- **-d (detached)**: Runs the containers in the background so you can continue using your terminal.

After a few seconds, check if the containers are healthy:

```
docker-compose ps
```

You should see:

- **sqlserver**: Marked as Up (healthy). (It takes about 10–20 seconds for the internal health check to pass while the database initializes).

  - Server name is sqlserver, username is sa, password is YourStrong!Pass123, and port is 1433. There are two databases there – adventureworks and awstaging.
  - To access this sql server from outside docker use port 2433

- **data_tools**: Marked as Up.
- **mssql-init**: Marked as Down – it is just an initial configuration script that restores AdventureWorks database from a backup file.

To get data into your system, you use **dlt**. It handles the connection to external databases (like AdventureWorks) and automatically creates the tables in your local SQL Server.

1. **Prepare your secrets**: Ensure your secrets.toml has the correct credentials for both the ext_sql_database (source) and local_mssql (destination). You can look at exemplar scripts in the app folder for such definitions
2. **Run the script**: Execute the Python script inside the data_tools container:

   ```
   docker exec -it data_tools python dlt_pipeline/load_sales.py
   ```

3. **Verify**: dlt will normalize the schema and load the tables into a dataset (schema) named AWStaging.

Once the data is in AWStaging, **dbt** takes over. It doesn't move the data; it sends instructions to SQL Server to transform it.

1. **The dbt Project**: Your code lives in the /app/dbt_project folder. This folder is "mounted," meaning changes you make in VS Code on your host machine appear instantly inside the container.
2. **Run a transformation**: To turn the raw tables into clean models, run:

   ```
   docker exec -it data_tools dbt run --project-dir /app/xxx --profiles-dir /app/xxx
   ```

Because we have defined volumes in your docker-compose.yaml, any data loaded into the sqlserver or files created in the data_tools /app directory will survive a docker-compose down. Your work is safe!

ASSIGNMENTS

The tasks will involve working with a local database and an external API (using Python) – utilising the solution developed as part of the previous set of tasks. In addition, external data sources (files) will be used to enrich the basic dataset (from the AdventureWorks database). The aim is to supplement and enrich the information available within the AdventureWorks operational database.

As part of the data supplementation, we will retrieve external data on product ratings and exchange rates. The former will allow us to enrich product information with the average, minimum and maximum product ratings. The latter will allow us to convert sales values into another currency – in this case PLN/EUR (to be selected).

As part of the data enrichment, we will create a new dimension representing the order date (OrderDate), broken down into individual date components, i.e. day, month, quarter and year. We will also add attributes representing the salesperson's full name, etc.

However, ultimately we want to create a simple star schema in the available SQL Server instance, within a new database, comprising 4 dimensions (Product, Salesperson, Sales Territory, Order Date) and 1 fact (Sales – at the product order level):

In this exercise, we will be using the following source data: the tables Production.Product, Production.ProductSubcategory, Production.ProductCategory, Sales.Salesperson, Sales.Territory, Person.Person, Person.CountryRegion, Sales.SalesOrderHeader, Sales.SalesOrderDetail; the file rating.csv; and the exchange rate table (for PLN or EUR).

TASK 1. SQL STATEMENTS

Prepare data structures that will allow you to implement a simple star schema. Specifically, we ultimately want to have four dimensions: Product, Salesperson, Sales Territory and Order Date (see the diagram above), as well as facts relating to individual products within an order. Consider which attributes will be needed and limit the set to the bare minimum. Remember to select the appropriate data types for the attributes.

1. Write an SQL statement to create a table for the Product dimension.
    a. Remember that you need primary keys and attributes relating to product ratings.
2. Write an SQL statement to create a table for the Salesperson dimension.
    a. Remember that primary keys and several additional attributes, such as the salesperson's full name, are required.
3. Prepare an SQL statement to create a table for the Sales Territory dimension.
    a. Remember that primary keys are required.
4. Prepare an SQL statement to create a table for the Order Date dimension.
    a. Remember that you need primary keys and several additional attributes, such as day, month (number and name), etc.
5. Prepare an SQL statement to create a table for the Sales facts

   a. Remember that you need foreign keys to the dimension tables and attributes representing sales values in a different currency.

6. Execute the commands on your SQL Server instance and the new (empty) database.

   a. NOTE – Ultimately, you are creating empty tables, which serve as data structures into which we will subsequently load appropriately cleaned and processed data.

As a solution, submit the individual SQL commands.

TASK 2. DATA EXTRACTION

Set up a new (empty) database and copy the source data into it; for example, use a dedicated data transfer tool (SQL Server Import and Export Data) to extract the basic data. Please note that you do not have to use this particular tool.

Place all newly created structures within the [Extract] schema – so that you can easily distinguish the temporary structures. At this stage, we want to separate the data extraction process from the logic of its processing, so try to transfer the data 'as-is', in its raw form.

1. Transfer data from the Production.Product, Production.ProductSubcategory, Production.ProductCategory, Sales.Salesperson, Sales.Territory, Person.Person, Person.CountryRegion, Sales.SalesOrderHeader and Sales.SalesOrderDetail tables.

2. Transfer data from the .csv files (rating). Ultimately, the data should be placed in the ProductRating table within the [Extract] schema.

3. Modify the Python script/function (from the previous task list) to allow data to be saved in the exchange rate table – the data from the table should enable sales values to be converted into PLN or EUR (as required). Ultimately, the data should be placed in the CurrencyRateData table within the [Extract] schema.

Please submit a brief description of the implementation and, if applicable, the source code (if required).

TASK 3. STAR SCHEMA – PRODUCT

Create a new Product table within the [Staging] schema in your own database (which already contains data within the [Extract] schema). The table should contain all the necessary product information (from all tables). As part of data cleansing, remember to handle empty values and other identified anomalies. As part of data standardisation, remember to prepare standardised values for individual attributes – e.g. harmonising units, harmonising flag names. As part of data enrichment, remember to supplement the data with calculated values derived from other attributes – e.g. the employee's full name based on first name, middle name and surname, the region name and postcode location from the postcode (and additional external data).

1. *Prepare an SQL query to create a dimension. There are several basic approaches:*
   a. *Using a SELECT INTO query and incorporating data processing logic within a single query*
   b. *Using an INSERT INTO query – similarly, this allows data processing to be incorporated within a single query*
2. *Using one of the above queries for pure data transport (possibly with basic elements of business logic for data processing) and then applying an ALTER TABLE command (or series of commands) to supplement the business logic of data processing (cleaning, standardisation, data enrichment).*
3. *Consider how the correctness of the above process can be tested – in terms of both data completeness and data accuracy.*
   a. *Suggest and briefly describe.*

TASK 4. STAR SCHEMA – TIME

1. *Within the [Staging] schema, create tables representing individual calendar days (one table row per day) from the earliest sale date to the latest sale date. Use the code below to create a table (Days) with a single column (DID) containing all dates from @start_date to @end_date:*

```
DECLARE @start_date [date] = CAST('2012-08-01' as [date]);
DECLARE @end_date [date] = CAST('2013-09-01' as [date]);
WITH D(ID) AS
( SELECT ROW_NUMBER() OVER(ORDER BY SalesOrderID) FROM
AdventureWorks20XX.Sales.SalesOrderHeader )
SELECT DATEADD(day, D.ID, @start_date) AS DID
INTO Staging.Days
FROM D
WHERE DATEADD(day, D.ID, @start_date) <= @end_date;
```

2. *Add additional attributes to the dimension to represent the day of the month, month number, month name, quarter, half-year and year. REMEMBER – T-SQL offers a whole range of functions for transforming data (see the additional resources at the top of the list).*

TASK 5. STAR SCHEMA –SALESPERSON AND SALES TERRITORY

As in Task 3, create a dimension for the SalesPerson and the SalesTerritory.

1. *Prepare SQL queries to create the dimension, using any of the approaches outlined in Task 3. Remember to take due account of the business logic involved in data processing (cleaning, standardisation, data enrichment).*
2. *Consider how the correctness of the above process can be tested – in terms of both data completeness and accuracy. Propose and briefly describe*

TASK 6 API DATA

Using data from an external API, we want to extend the model from the previous task list to include the ability to express sales figures (for AW) in PLN (sales figures are currently in USD, and we would like to convert them to PLN – we are only interested in the order amounts). To do this, we will use the function from the previous task and the prepared auxiliary data structure (the CurrencyRateData table).

Create a script that performs the following functions:

1. *Retrieve the required data from the API into the CurrencyRateData table. The table will contain information on average USD exchange rates against the Polish zloty for a period of your choice (depending on the data available in the database) – let this be a minimum of 2 years. For days for which no rates are provided (such as weekends), fill in the data using the value from the first previous day for which a value is available.*

Save the script as part of the report.

(*) TASK 7. DATA MODEL ENRICHMENT AND VISUALISATION

Using the auxiliary table, populate the model with the required values in PLN. Then prepare a set of visualisations so that BI system users can utilise the prepared data model.

1. *Add an additional column to the fact table representing the transaction amount expressed in PLN. Then add information to the model indicating whether the exchange rate rose or fell on the order date compared to the previous day. Consider how to modify the model.*
2. *Create a new report containing:*
    a. *a chart showing total sales (the sum of all completed transactions) expressed in the transaction currency (USD) and in PLN (converted at the exchange rate on the day of sale). Create the chart for the time selected when completing Task 4. Remember to include a legend and label the axes and the chart.*

b. *a chart allowing you to compare sales (the sum of all completed orders) against the exchange rate trend (upward/downward). Create the chart for the time period selected when completing Task 4. We simply want to check whether sales are higher on days when the currency falls. Remember to include a legend and label the axes and the chart.*

Describe the process for completing points 1 and 2 (include code snippets); save screenshots of the visualisation as part of the report.

(*) DUCKDB – THE LIGHTWEIGHT ROLAP APPROACH

***DuckDB** is a columnar relational database management system (RDBMS) designed specifically for analytics and OLAP (Online Analytical Processing) query processing. It is lightweight, embedded, and highly efficient for data analysis scenarios on a single machine.*

*A key feature of the DuckDB solution is the **absence of an external server**. This means it runs **in-process** and does not require the installation or configuration of a separate database server. While this has its own pros and cons, DuckDB itself utilizes a **columnar data format**, meaning data is stored by columns. This significantly accelerates aggregation, filtering, and grouping operations (ideal for analytical SQL). Consequently, it is optimized for single-node analysis, often outperforming Pandas and SQLite in aggregation tasks. Additionally, DuckDB supports SQL with extensions and provides full support for standard SQL, including window functions, CTEs, subqueries, JOINs, and analytical extensions. This allows for easy integration with various data visualization tools. Furthermore, it provides integration with external data, for example, allowing for the direct reading of **CSV, Parquet, and JSON** files without the need to import them into the database. All of this is part of a minimal, almost zero-configuration approach—once installed (a single library), you can immediately create databases in memory or on disk.*

Installation

The simplest way to install DuckDB is:

```
pip install duckdb
```

You can then use DuckDB from Python. Example of using DuckDB to fetch data from an external CSV file:

```
import duckdb

# Connect to an in-memory database
conn = duckdb.connect()

# Execute a query on a CSV file without loading it entirely
result = conn.execute("""
    SELECT
        department,
        AVG(salary) as avg_salary,
        COUNT(*) as employee_count
FROM 'employees.csv'
GROUP BY department
ORDER BY avg_salary DESC
""").fetchdf()  # returns a pandas DataFrame

print(result)
```

*DuckDB also allows for **persistent data storage**. The function duckdb.connect(dbname) establishes a connection to a persistent database file. Below is an example:*

```
import duckdb

# Create a connection to a file called 'file.db'
con = duckdb.connect("file.db")

# Create a table and load data into it
con.sql("CREATE TABLE test (i INTEGER)")
con.sql("INSERT INTO test VALUES (42)")
```

```
# Query the table
con.table("test").show()

# Explicitly close the connection or use a context manager
con.close()
```

Typical Use Cases

1. ***Direct File Operations:*** *Querying files without creating tables (e.g., SELECT * FROM 'sales_*.parquet').*
2. ***Joining Diverse Data Sources:*** *Joining a CSV with a Parquet file in a single SQL view.*
3. ***Exporting Results:*** *Direct export to Parquet or CSV using write_parquet() or write_csv().*
4. ***Pandas Integration:*** *Converting query results directly to a DataFrame using .df().*

DUCKDB AND THE STAR SCHEMA

The star schema is the foundation of data modeling in analytical systems, being the most intuitive way to organize information for business purposes. Its name is derived directly from its visual layout, where a central table, called a ***fact table****, is surrounded by smaller* ***dimension tables****, connected directly to it like the arms of a star. This structure is primarily designed to simplify SQL queries and optimize performance in OLAP-type systems. In the era of modern columnar databases like DuckDB, the star schema takes on special significance because it allows the analytical engine to lightning-fast filter and aggregate massive datasets with minimal computational overhead.*

The Fact Table: The Core of the Star

The central point of the schema is the Fact Table, which stores measurable, quantitative data regarding specific business events, such as sales, user logins, or sensor readings. Each row in the fact table represents a specific fact and typically contains two groups of columns: ***foreign keys*** *connecting it to dimensions and* ***numerical measures*** *that are subject to summation or averaging. A key decision when designing a fact table is determining its* ***grain****, or the level of data detail. For example, the grain could be a single transaction at a checkout or a daily sales report. A properly selected grain allows for a balance between performance and the ability to drill down into details during analysis.*

Dimension Tables: Providing Context

The fact table is surrounded by a context of data—dimension tables. While facts answer the question "how much," dimension tables provide answers to "who," "where," "when," and "what". They are a collection of descriptive attributes that define the business context for the numbers contained in the fact table. For instance, a product dimension might contain information about its name, category, brand, and color. Dimension tables are characterized by a high number of text columns and fewer rows compared to the fact table. An important feature of this model is the ***denormalization*** *of dimensions, which means intentionally repeating certain information within the table to avoid complex joins between multiple tables, significantly speeding up report generation.*

Advantages in Analytical Applications

The advantages of the star schema in analytical applications include:

- ***Simplicity and Intuitiveness:*** *The star schema reflects how business users perceive their data. When an analyst asks about revenue by product category per region, they intuitively think in terms of dimensions (time, product, location) and measures (revenue), making it easier to write queries and understand the model.*
- ***Query Performance:*** *The schema is optimized for analytical queries. Due to fewer tables and direct connections, queries that slice data by dimensions typically run faster. Modern columnar databases like DuckDB process aggregations and joins with extraordinary speed.*
- ***BI Tool Compatibility:*** *Most BI tools (Tableau, Power BI, Looker, etc.) are designed with the star schema in mind. They can often automatically detect relationships between facts and dimensions, facilitating report creation for end users.*

Modern Implementation and Performance

Although star schemas originated in the era of row-based data storage, they have gained new life through modern columnar engines. Contemporary solutions like DuckDB utilize the specifics of the star schema to achieve

*extreme analytical acceleration. Because data in fact tables is usually ordered by time or dimension keys, columnar engines can apply techniques such as **query vectorization** and **data skipping**.*

*In a star schema, queries become predictable—usually boiling down to filtering dimension tables and joining them to the fact table. This structure is significantly more efficient than the **snowflake schema**, which, through excessive normalization of dimensions, forces the processor to perform many costly JOIN operations that often become a bottleneck in modern systems. An additional advantage is **data compression**; dimension keys (usually integers) in the fact table compress very well, reducing storage space and I/O load. All of this is achieved within a simple model, eliminating the need for complex "One Big Table" (OBT) flattening, which can lead to massive data duplication and management issues while maintaining high query performance.*

Compromises and Limitations

Naturally, no system is perfect, and the star schema involves certain trade-offs:

- ***Data Redundancy:** Because data is denormalized, repeated values are stored. If 100 products belong to the same category, the category name is saved 100 times, increasing storage requirements.*
- ***Data Maintenance:** Updating values in denormalized tables requires more care. For example, if a brand name changes, it must be updated in many product rows, making robust ETL/ELT processes crucial.*
- ***Unsuitability for OLTP:** The star schema is optimized for read-intensive analytical workloads (OLAP) rather than frequent transactional writes (OLTP). Its structure and redundancy do not meet the needs of applications requiring fast insertions and updates with strict normalization.*

CREATING A STAR SCHEMA IN DUCKDB

We will begin the process of creating a star schema by defining a simple business scenario: analysing sales in a retail store. Our aim is to create a structure that will allow us to quickly check revenue by date, product and store location.

First, we create dimension tables to store the unique attributes of our entities. Note the lack of normalisation – in the products table, we store both the name and the category.

```
-- Creating a Product dimension
CREATE TABLE dim_product (
    product_key INTEGER PRIMARY KEY,
    product_name VARCHAR,
    category VARCHAR,
    brand VARCHAR,
    price DECIMAL(10, 2)
);

-- Creating a Shop dimension
CREATE TABLE dim_store (
    store_key INTEGER PRIMARY KEY,
    store_name VARCHAR,
    city VARCHAR,
    region VARCHAR
);

-- Creating the Dimension of Time
CREATE TABLE dim_date (
    date_key INTEGER PRIMARY KEY,
    full_date DATE,
    year INTEGER,
    quarter INTEGER,
    month INTEGER,
    day_of_week VARCHAR
);
```

The fact table links our dimensions using foreign keys and stores numerical values (quantity and amount). In this example, the granularity is a single sale transaction for a specific product in a given shop on a given day.

```
CREATE TABLE fact_sales (
    sale_id INTEGER PRIMARY KEY,
    date_key INTEGER REFERENCES dim_date(date_key),
    product_key INTEGER REFERENCES dim_product(product_key),
    store_key INTEGER REFERENCES dim_store(store_key),
    quantity INTEGER,
    total_amount DECIMAL(12, 2)
);
```

Thanks to this structure, querying total sales by product category in individual regions becomes simple and efficient:

```
SELECT
    p.category,
    s.region,
    SUM(f.total_amount) AS total_revenue
FROM fact_sales f
JOIN dim_product p ON f.product_key = p.product_key
JOIN dim_store s ON f.store_key = s.store_key
GROUP BY p.category, s.region
ORDER BY total_revenue DESC;
```

For those interested:

- *https://motherduck.com/learn/star-schema-data-warehouse-guide/*
- *https://thinhdanggroup.github.io/duckdb/*

CONNECTING DUCKDB TO POWER BI

Since DuckDB is an "in-process" engine, connecting to Power BI differs from standard servers. There are three main methods:

1. ***Via ODBC Driver (Most stable for reports):***
    - *Install the official **DuckDB ODBC Driver**.*
    - *Configure a Data Source Name (DSN) in the Windows ODBC Data Source Administrator (64-bit), pointing to your .db file.*
    - *In Power BI, use **Get Data -> ODBC** and select your DSN.*
2. ***Via Python Script (Ideal for prototyping):***
    - *Use the **Get Data -> Python Script** option.*
    - *Paste a script that connects to the database via duckdb.connect(), executes SQL, and returns a Pandas DataFrame.*

```
import duckdb
import pandas as pd

# Connecting to a database file
con = duckdb.connect('C:/sciezka/do/twojej_bazy.db')

# Loading data into a Pandas frame
df = con.execute("SELECT * FROM sprzedaz_agregaty").df()

# Ending the call
con.close()
```

    - Power BI will run the script, launch the DuckDB engine within the process, and import the query result as a table.
3. ***Via Custom Connector:***

- *Use the community-developed Power Query connector available on GitHub (e.g., duckdb-power-query-connector).*

Note: *DuckDB typically locks the database file for a single process. If the file is open in another tool, Power BI may show a connection error. Use read_only=True settings where possible.*